

EE8257 Information Security



Lecture 5

Asymmetric / Public Key Cryptography

P. S. Ranaweera

E313

Department of Electrical and Information Engineering

Faculty of Engineering

University of Ruhuna



MOST IMPORTANT TOPIC



Outline

- Overview
- Uses of Public Key
- Public Key Algorithms
 - RSA
 - DH
 - ECC
- ECC based DH
- Public Key Notations
- Sign-and-Encrypt
- Encrypt-and-Sign

the reason to shift from private to public is that it is needed to rely on trusted third party

Public Key Crypto Overview

- 2 keys are used
 - **Public key** to encrypt : known to all 
 - **Private key** to decrypt : known only by the recipient
- Overcome the issue of distributing the key securely as in symmetric key crypto
- Founded by GCHQ (British) in 1960s
- Crypto system has a complex mathematical structure
- Crypto is based on a **one-way trap door function**
 - Easy to compute in one direction
 - Hard to compute in the other direction (Reverse Engineering in almost infeasible)
 - E.g.: If p, q are very large, $N = pq$: It would be difficult to find N with given p, q 

Uses of Public Key

- Encryption
 - If a message is encrypted with Alice's ***public key*** (which is known to everyone), only Alice's ***private key*** can decrypt it
- Digital Signature
 - Reverse process of encrypting
 - Sign : encrypting with ***private key***
 - Verify : decrypting with ***public key***
 - Signing can be done only by ***private key*** holder → Signature
 - Conformation of identity → Non-Repudiation

Public Key Algorithms



- RSA
- Diffie-Hellman
- Elliptic Curve Crypto
- ECC Diffie-Hellman





RSA

Rivest Shamir Adleman



- RSA concept was invented by Cliff Cocks (GCHQ)
- Rivest, Shamir and Adleman of MIT has reinvented it
- **Algorithm :**

- Choose two large prime numbers p and q
- Compute $N = pq$
- Compute $\phi(N) = (p-1)(q-1)$
- Choose e **relatively prime** to $\phi(N) \Rightarrow \gcd[e, \phi(N)] = 1$
- Determine the multiplicative inverse of e w.r.t mod $\phi(N)$
 - i.e. : Find d , s.t. $\rightarrow$

go with the lowest prime number

we are searching for a number which has no common divisors with 20.
it can have many numbers

torsion value

$$ed = 1 \bmod \phi(N) = 1 \bmod (p-1)(q-1)$$

see the example in the book

RSA Algorithm

- N is considered as the **Modulus**
- e encryption exponent
- d decryption exponent
- **Public Key** = (N, e)
- **Private Key** = (N, d) here this is mod remeber this, most people get confusions 
- Encryption $\rightarrow C = M^e \bmod N$
- Decryption $\rightarrow M = C^d \bmod N$ 
- Factoring would break RSA $\rightarrow$ Determining p and q from N

RSA Efficiency and Vulnerability

- RSA efficiency could be boosted by having the same **encryption exponent** e for all the parties
 - Most commonly used way $\rightarrow e = 3$  
 - In this case If, $M < N^{1/3} \rightarrow$ RSA vulnerable to  **cube root attack**
 - $C = M^e = M^3$
 - This attack is avoided by padding the M to make $M > N^{1/3}$
- Another approach to speed up RSA is to have $e = 2^{16} + 1$
 - Vulnerability exist if only same encrypted message is sent to $2^{16} + 1$ users

chinese remainder theorem
we dont go for computation



Diffie-Hellman (DH)

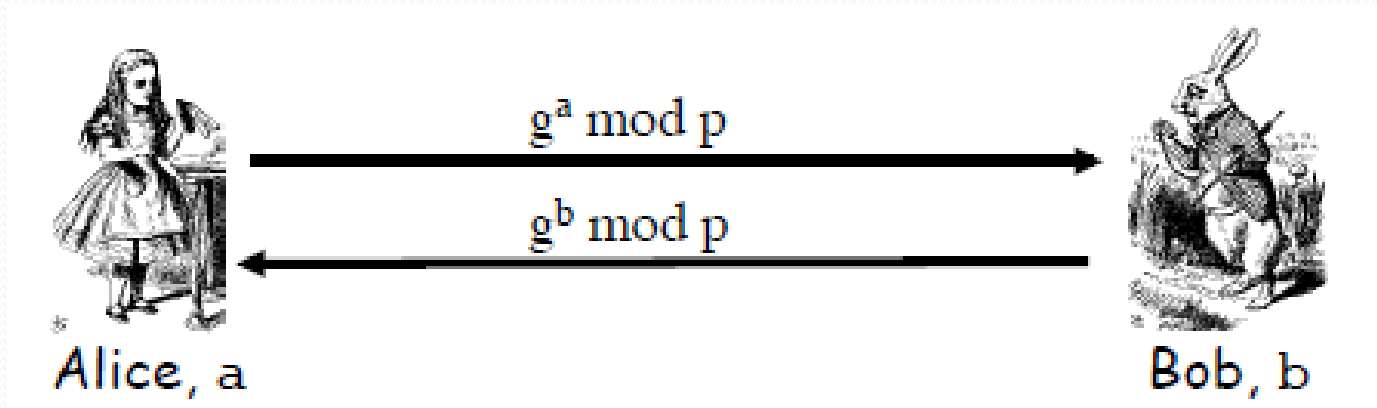
Diffie-Hellman (DH) invented in stanford

- Invented by Malcolm Williamson (GCHQ)
- Reinvented by Whitefield Diffie and Martin Hellman (Stanford)
- It is a key exchanging algorithm
 - Share a secret $\rightarrow$ used to share the symmetric key
 - Not used for encrypting or signing
- Security in DH rely on the computational difficulty in ***discrete log problem***
 - If g and p are given, If $x = g^k \rightarrow \log_g(x) = k$
 - If $x = g^k \bmod p$, $\rightarrow \log_g(x) \neq k \leftarrow$ Finding k is difficult

DH Algorithm

- p is selected as a prime and g as a generator(arbitrary)
- Alice generate Alice's secret a
- Bob generate Bob's secret b
- Alice send Bob $\rightarrow g^a \bmod p$
- Bob sends Alice $\rightarrow g^b \bmod p$
- Both compute the shared secret $\rightarrow g^{ab} \bmod p$
- The secret is used as the symmetric key

DH Process



- Alice compute : $= (g^b)^a \bmod p = g^{ab} \bmod p$
 - Bob compute : $= (g^a)^b \bmod p = g^{ab} \bmod p$
- } Used as the symmetric key

DH security

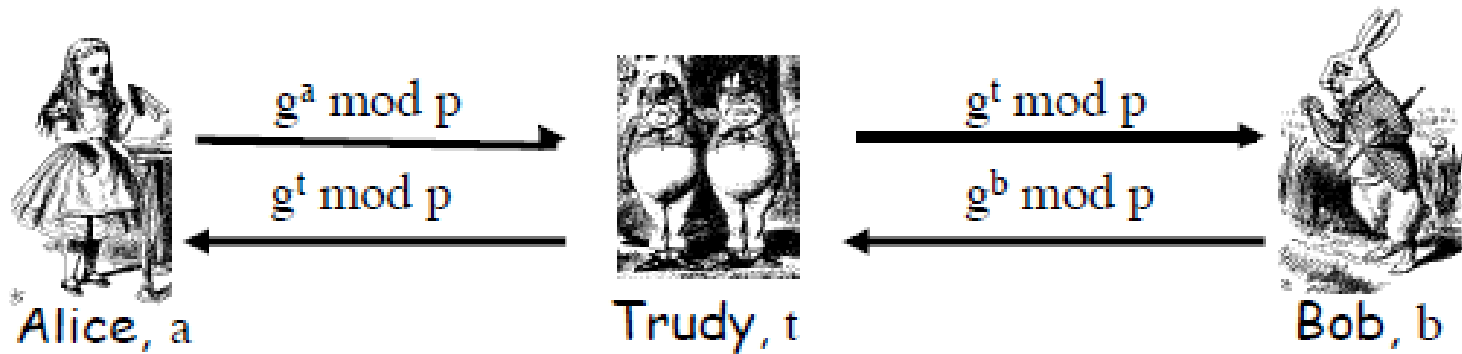
[see the notes](#)

- If Alice and Bob are using $K = g^{ab} \bmod p$, as the symmetric key,
- Trudy can extract independent components $g^a \bmod p$ and $g^b \bmod p$
- But Trudy cannot compute $g^{ab} \rightarrow g^a \cdot g^b = g^{a+b} \neq g^{ab}$
- Trudy must find a and b separately
 - Trudy must solve discrete log problem

DH MiM Attack

- Man in the Middle (MiM) Attack :

alice thinks trudy as bob and bob thinks trudy as alice



- Trudy and Alice communicate with $g^{at} \bmod p_{(g^t)^a \bmod p}$
- Trudy and Bob communicate with $g^{bt} \bmod p_{(g^t)^b \bmod p}$
- Alice and Bob doesn't know about Trudy →
 - Confidentiality and Integrity is compromised

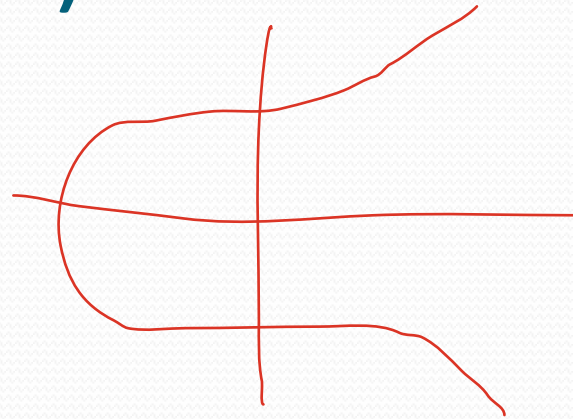
Avoid DH MiM attack

- Encrypt DH exchange with a symmetric key
 $E(g^a \bmod P)$
- Encrypt DH exchange with a public key pairs
encrypt $g^a \bmod p$ with Bobs public key then only Bob can decrypt this
- Signing DH values with private key pairs



Elliptic Curve Cryptography (ECC)

this is an elliptic curve all the points must
have on this curve



Elliptic Curve Cryptography (ECC)

- ECC is not a cryptosystem
- ECC provides an approach to perform mathematical operations in an algorithm differently
- ECC is used in already discussed algorithms
 - DH
 - RSA
- Fewer bits are required for same level of security (more efficient) 
- But mathematical operations costs more → high complexity
- Efficiency is valued in environments with lesser resources on capacity → Mobile devices

there is a combined way to use in these algorithms with ecc

ECC
256
384
512

RSA
3072
7680
15360

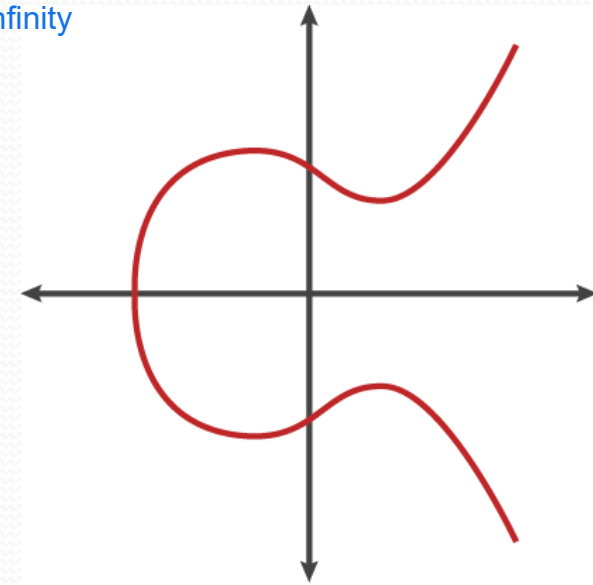
Elliptic Curves

two ends goes to infinity

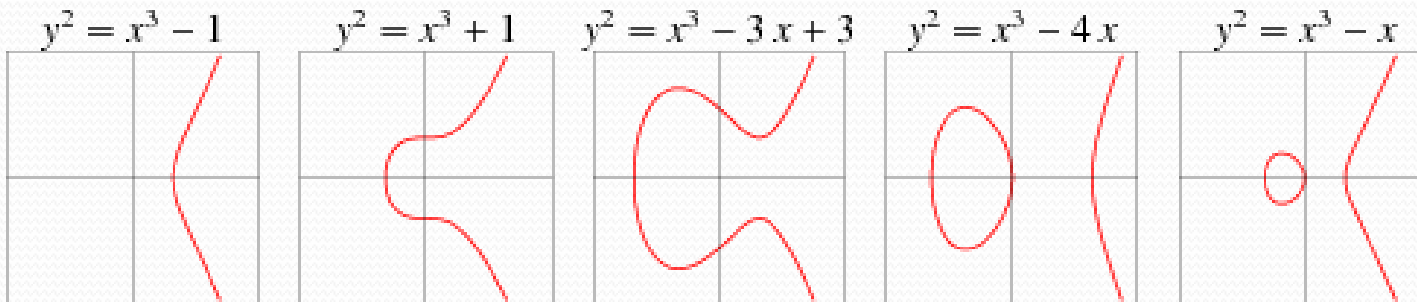
- Elliptic Curves takes the form of a function E;

$$E : y^2 = x^3 + ax + b$$

- It also has a '*point at infinity*'
- Lot of mathematics are involved with elliptic curves



we only deals with addition in elliptic curve



if we want to take the addition of the same point ($p+p$) draw a tangent and find the point that intersect the curve. then take the mirror image

every point we are dealing with lies on the curve

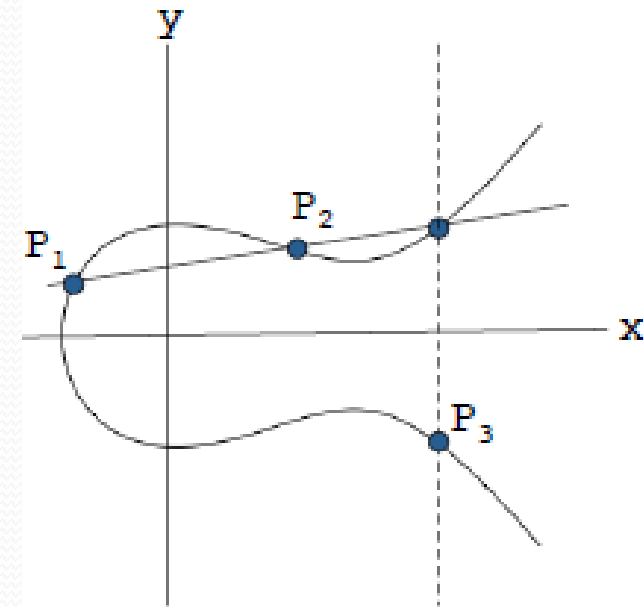
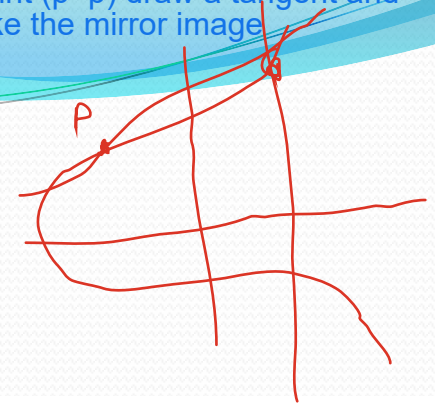
Elliptic Curves (Cont...)

- Consider points P_1, P_2, P_3 lying on the elliptic curve
- Addition of two points :

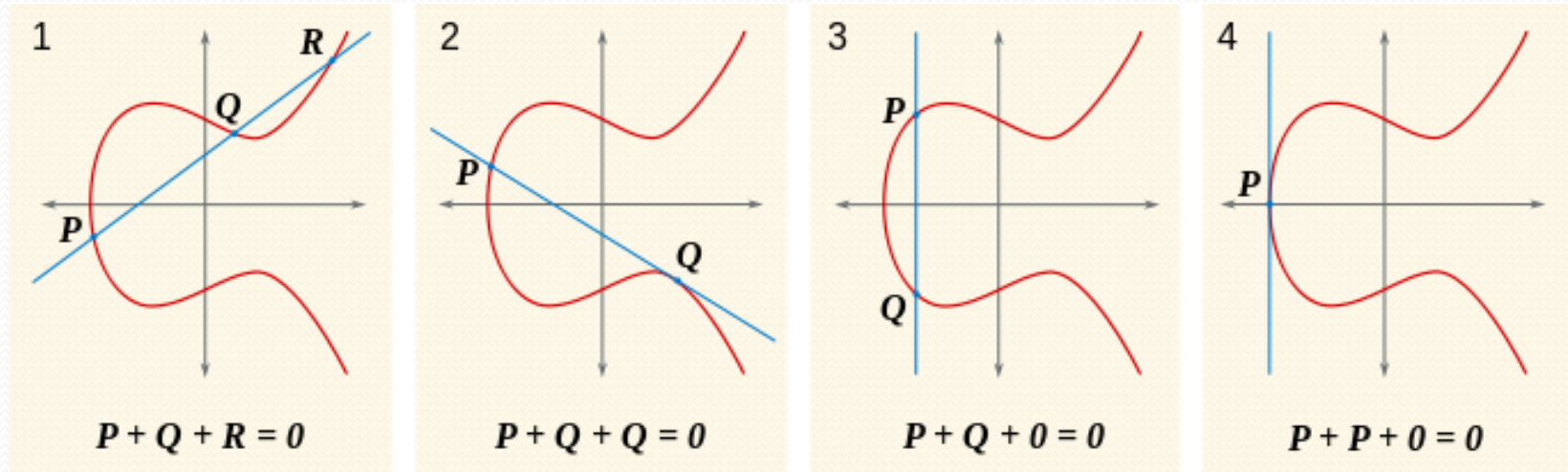
$$P_3 = P_1 + P_2$$

- Reflection on X-axis of the intersection point of the extended P_1P_2 line
- Addition is enough...!!!

we are drawing a line from p_1 to p_2 until it intersect the curve again then take the mirror image of that point

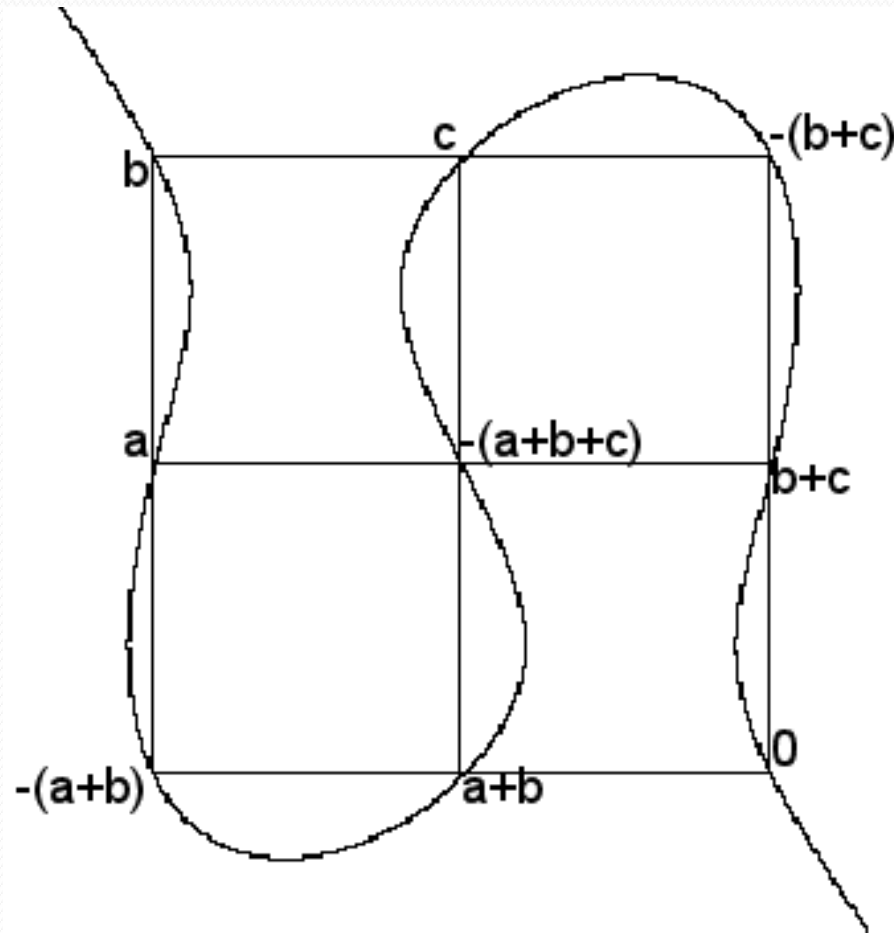


Properties of Elliptic Curves



this is like $-q + q$

Properties of Elliptic Curves (Cont...)

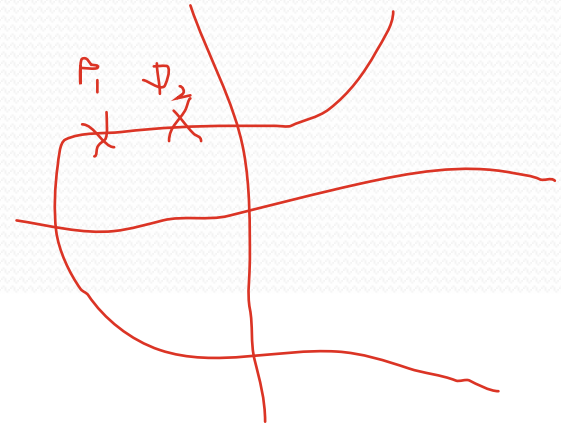


Modular into ECs

- Modular is used for ECs to restrict the range of values
→ to have a discrete set of points
- Modular EC function → $y^2 = x^3 + ax + b \pmod{p}$
- If we consider the solution of the function $y^2 = x^3 + 2x + 3 \pmod{5}$
 - $x = 0; y^2 = 3 \rightarrow \text{no solution (mod 5)}$
 - $x = 1; y^2 = 6 = 1 \rightarrow y = 1, 4 \pmod{5}$
 - $x = 2; y^2 = 15 = 0 \rightarrow y = 0 \pmod{5}$
 - $x = 3; y^2 = 36 = 1 \rightarrow y = 1, 4 \pmod{5}$
 - $x = 4; y^2 = 75 = 0 \rightarrow y = 0 \pmod{5}$
- Point on the elliptic curve → $(1,1)(1,4)(2,0)(3,1)(3,4)(4,0)$ and ∞

the is definetely coming for exam

Addition on ECs



$$P_1 = (x_1, y_1), P_2 = (x_2, y_2)$$

$$P_1 + P_2 = P_3 = (x_3, y_3) \text{ where}$$

$$x_3 = m^2 - x_1 - x_2 \pmod{p}$$

$$y_3 = m(x_1 - x_3) - y_1 \pmod{p}$$

$$\text{And } m = (y_2 - y_1)(x_2 - x_1)^{-1} \pmod{p}, \text{ if } P_1 \neq P_2$$

$$m = (3x_1^2 + a)(2y_1)^{-1} \pmod{p}, \text{ if } P_1 = P_2$$

Special cases: If m is infinite, $P_3 = \infty$, and

$$\infty + P = P \text{ for all } P$$

we are dealing with the case where $p_1=p_2$ (tangent case)

EC Addition Example

this part is important

- Try the addition of two points $(1,4), (3,1)$ on the elliptic curve $y^2 = x^3 + 2x + 3 \pmod{5}$

$$P_3 = P_1 + P_2 = (1,4) + (3,1) = (x_1, y_1) + (x_2, y_2) = (x_3, y_3)$$

$$P_1 \neq P_2 \Rightarrow m = (y_2 - y_1)(x_2 - x_1)^{-1} \pmod{p}$$

$$m = (1 - 4)(3 - 1)^{-1} \pmod{5} = -3 \cdot 2^{-1} \pmod{5}$$

$$= (-3 \pmod{5})(2^{-1} \pmod{5}) = 2 \times 3 \pmod{5} = 6 \pmod{5} = 1$$

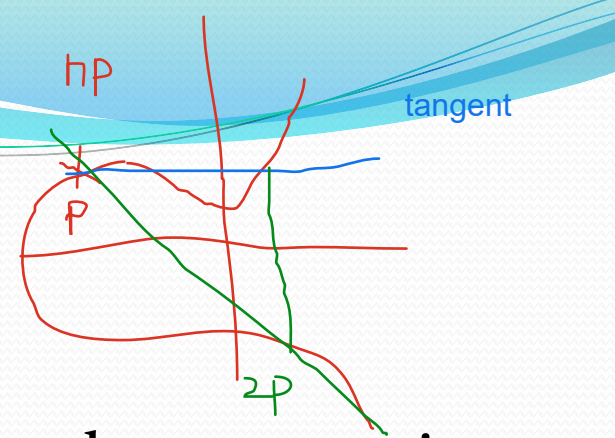
$$x_3 = m^2 - x_1 - x_2 \pmod{p} = (1 - 1 - 3) \pmod{5} = 2 \pmod{5}$$

$$y_3 = m(x_1 - x_3) - y_1 \pmod{p} = [1(1 - 2) - 4] \pmod{5} = 0 \pmod{5}$$

$$P_3 \equiv (x_3, y_3) = (2, 0)$$

- Try for $(3,1), (3,4)$ and $(3,1), (3,1)$

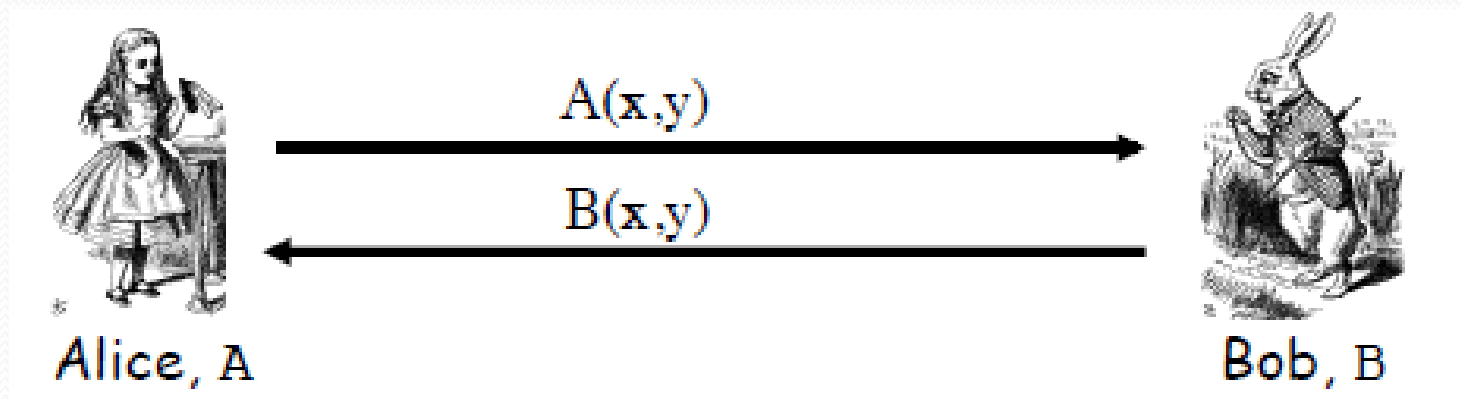
EC Multiplication



- Multiplication of ECs is done by adding the same point consecutively
- For an instance $\rightarrow A(x, y) = \underbrace{(x, y) + (x, y) + (x, y) + \dots + (x, y)}_A$
- $21(3, 2) = \underbrace{(3, 2) + (3, 2) + (3, 2) + \dots + (3, 2)}_{21}$
we adding the same point the amout of times it is needed to be multiplied.
- But the point must be on the curve

ECC Diffie-Hellman

- Public : Elliptic curve function and the point been considered
- Secret : Alice and Bob's secrets as in DH



- Alice Compute $\rightarrow A(B(x, y))$
 - Bob Compute $\rightarrow B(A(x, y))$
- These two expressions are grouped by a large right-facing curly bracket, followed by the equation $A(B(x, y)) = B(A(x, y))$. An upward-pointing arrow from the word "Key" below points to the result of the computation.

ECC DH Example

- Elliptic Curve : $y^2 = x^3 + 11x + c \pmod{167}$
- Find c for a considered point $(2,7)$ $\rightarrow c = 19$
- Elliptic Curve : $y^2 = x^3 + 11x + 19 \pmod{167}$ 
- If Alice's and Bob's secret are $A = 15$ and $B = 22$
- Alice's secret $\rightarrow A(2,7) = 15(2,7) = (102,88)$
- Bob's secret $\rightarrow B(2,7) = 22(2,7) = (9,43)$
- Alice and Bob shares the values
- Alice's computation $\rightarrow A(9,43) = 15(9,43) = (131,140)$
- Bob's computation $\rightarrow B(102,88) = 22(102,88) = (131,140)$
- Symmetric Key $\rightarrow (131,140)$

Security in ECC DH

- Same security concepts apply as in DH and ECs have enhanced the security
- This version of DH is difficult to break than the regular DH
- Smaller values of lesser number of bits are resulted for keys and encrypted texts
- ECC DH is still subjected to MiM attack

if you put the signature anyone can read it if only encryption it is secure

Public Key Notations

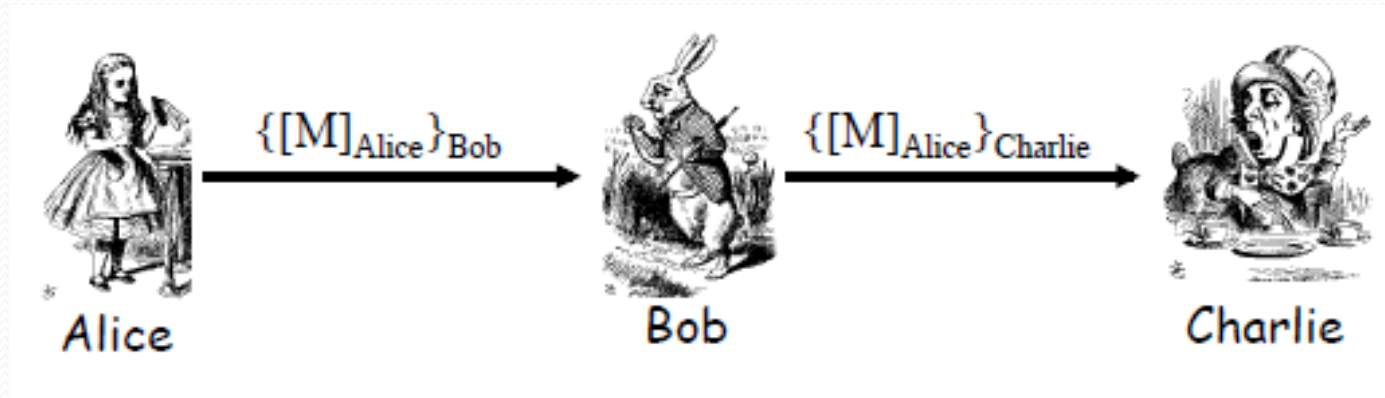
- If message and ciphertext are denoted by M and C
if you are using public key use {}
if you are using private key use []
Cipher text
Message
- Encryption with Alice's public key $\rightarrow C = \{M\}_{Alice}$
whose key you are using
- Decryption with Alice's private key $\rightarrow M = [C]_{Alice}$
- Signing with Alice's private key $\rightarrow S = [M]_{Alice}$
you are getting a signature not a cipher text
- Un-signing (verifying) with public key $\rightarrow M = \{S\}_{Alice}$
this is not secure
$$\left[\{M\}_{Alice} \right]_{Alice} = \underbrace{\{ [M]_{Alice} \}}_{original\ message} = M$$

Sign-and-Encrypt Approach

- First sign the message and encrypt

- Encryption ensure confidentiality
- Signing ensure integrity protection 

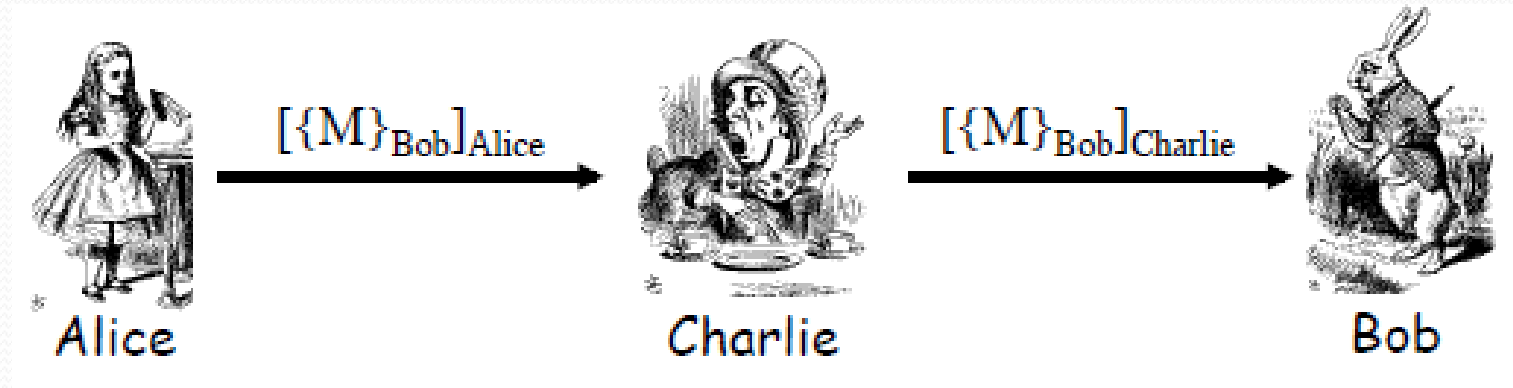
$$\{[M]_{Alice}\}_{Bob}$$



- Only between trusted parties : cannot stop the misuse of the signature

Encrypt-and-Sign Approach

- First encrypt then sign $\rightarrow [\{M\}_{Bob}]_{Alice}$



- Not suitable
- Suitable when information is transmitted over many peers
- But this process is very costly : Solution $\rightarrow$ Hybrid Crypto System